

Nombre: Pedro Fernando
Apellidos: Sanchez Uribe
Código: 19130052

1

1. El repositorio es la **SSOT**, porque toda la aplicación obtiene los datos exclusivamente a través de él, sin importar si vienen de Room.

* ¿Cómo coordina Room y Retrofit?

Según recuerdo son 3 etapas

1. Emisión local inmediata; el repositorio consulta Room y emite los datos cacheados instantáneamente a la UI, evitando pantallas de carga vacías.
2. Dispara una petición HTTP vía Retrofit al backend
3. Si la respuesta es exitosa de la red, los datos se guardan en Room, la Room detecta el cambio en la tabla y propaga únicamente la actualización al Flow que la UI está observando.

2. * **ViewModel Scope** es un Coroutine Scope provisto por el framework de Android, que está vinculado al ViewModel.

* **Fundamental**: gestiona la memoria para evitar que los procesos queden "huérfanos" corriendo en segundo plano y referenciando objetos que ya no existen, lo que causaría memoria leaks.

* **Usuario decide cerrar pantalla**

Android destruye el ViewModel y llama a `onCleared()`. El ViewModel se cancela. La coroutine que esperaba la respuesta de red se detiene de forma segura sin actualizar ningún estado de UI.

3

	Flow (Frio)	StateFlow (Caliente)
Activación	Solo cuando hay collect activo	Siempre activo
Valor Actual	No tiene ninguno	Siempre tiene lo último
Nuevo suscriptor	Recalcula desde cero	Recibe el último valor
Recursos	No consume si nadie escucha	Siempre en memoria
Uso típico	Transformaciones internas	Estado UI en ViewModel
Emisiones	Todas las del flujo	Solo cuando el valor cambia

* ¿Porque se prefiere StateFlow vs Flow convencional?

Porque la UI necesita siempre tener un estado disponible, por eso se prefiere StateFlow.

④ La DI manual consiste en usar la clase `Application` como contenedor de dependencias. Como Android la instancia una sola vez durante toda la vida del proceso, se declara ahí los objetos de infraestructura (`ROOM`, `retrofit`) y los `view models` los acceden mediante `application as My app`. `repository` garantizando que todos comportan la misma instancia si usas `Hilt Dagger`.

* Para que sirve Lazy?

Es un delegado de Kotlin que explica inicialización diferida: el objeto no se construye al arrancar la `Application`, sino la primera vez que accede a él. Ese resultado queda cacheado y todas las llamadas posteriores devuelven la misma instancia.

II

1. El `Stylus` de datos que emite automáticamente cada vez que una Tabla de `Room` cambia se implementa usando el tipo `Flow`.
2. El operador `Dispatchers`. `Default` permite ejecutar tareas pesadas fuera del `Main Thread`, específicamente optimizado para tarea de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo `location`.
4. La función `by lazy` de la clase `Application` es la que garantiza que los componentes comportan la misma instancia del repositorio mediante un `call`.
5. El compilamiento que permite transformar una API basada en callbacks antiguos es una función suspendible moderna llamada exactamente `suspend Coroutine`.
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente `AndroidManifest.xml`.

7. En la arquitectura AOSP, la capa que actúa como puente entre el framework y los controladores se denomina HAL o Hardware Abstraction Layer

8. El operador de Flow que permite combinar múltiples eventos (como GPS y Sensores) para emitir un estado unificado es combine()

III

I. Repositorio y DI

```
class UsuarioRepository (private val usuarioDao : UsuarioDao) {
```

```
    fun getUsuarios() = usuarioDao.getAll()
```

```
    suspend fun insertUsuario(usuario : Usuario) {
        usuarioDao.insert(usuario)
    }
```

```
}
```

```
class MyApp : Application() {
```

```
    val database : AppDatabase by lazy {
        Room.databaseBuilder (this, AppDatabase::class, "app - database")
            .build()
    }
```

```
}
```

```
    val usuarioRepository : UsuarioRepository by lazy {
        UsuarioRepository (database, usuarioDao())
    }
```

```
}
```


②

// a)

```
data class Usuario(  
    val nombre: String,  
    val apellidos: String,  
    val edad: Int?)  
)
```

// b)

```
val usuario = Usuario(  
    nombre = "Pedro",  
    apellidos = "Sánchez Uribe",  
    edad = 25)
```

// c)

```
val letrasNombre = usuario.nombre.length ?: 0  
val letrasApellidos = usuario.apellidos.length ?: 0  
val puntajeIdentidad = letrasNombre + letrasApellidos
```

// d)

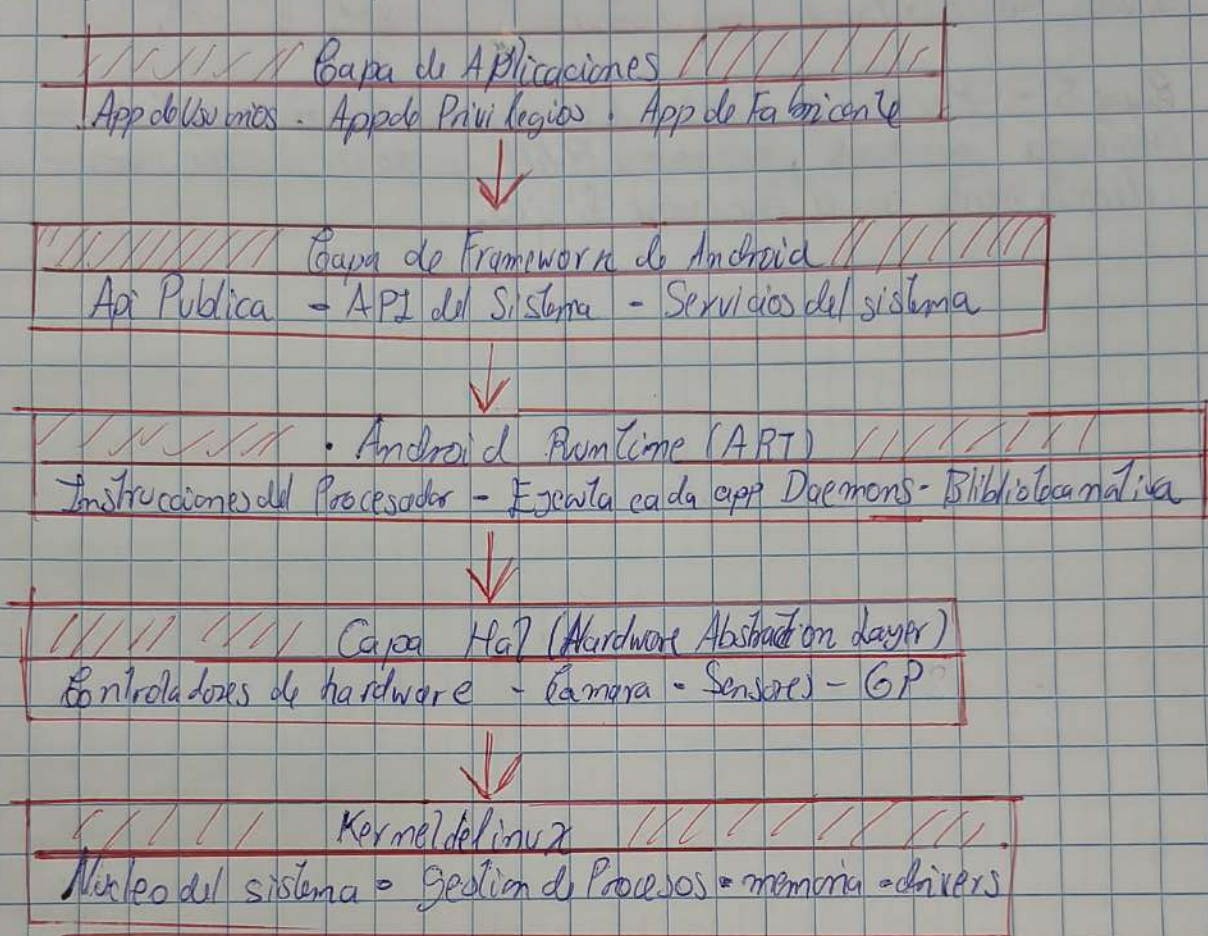
```
val nombreCompleto = "${usuario.nombre} ${usuario.apellidos}"  
println("Usuario: $nombreCompleto, tu puntaje de identidad es :  
$puntajeIdentidad")
```

// e)

```
val nombreSeguro = usuario.nombre ?: ""  
val apellidosSeguro = usuario.apellidos ?: ""  
val puntajeSeguro = (nombreSeguro.length) + (apellidosSeguro.length)  
println("Usuario: $nombreSeguro, tu puntaje de identidad es :  
$puntajeSeguro")
```


IV

1 Diagrama de 5 Capas.



2 Identificación Total.

Capa 1 - Aplicaciones: Capa donde residen todas las apps del usuario, OEM y fabricante que interactúan directamente en el sistema.

Capa 2 - Framework de Android: Provee la API pública y los servicios del sistema (como Surface Flinger y Media Service) que los desarrolladores usan para construir apps.

Capa 3 - Android Runtime (ART): Convierte el bytecode .dex en instrucciones ejecutables para el procesador; también aloja los daemons y bibliotecas nativas como init, logd y libe.

Capa 4 - HAL (Hardware Abstraction Layer): Actúa como puente estándar entre el framework de Android y los controladores físicos del hardware como cámaras, sensores y GPS.

Capa 5 - Kernel de Linux: Núcleo del sistema operativo que gestiona procesos, memoria RAM y drivers para comunicarse directamente con el hardware físico.